

Module 8 Lab:

Katalon Testing Studio & AI Integration

JR Clark (002035589)

NEU CSYE6255 (Summer 2026)

Professor Omar Badreldin / TA Karthik Kashyap

PART 1: Technical Implementation & Coverage

This lab required implementing four major testing types in Katalon Studio—Web/Cloud, API, Mobile, and Desktop—using multiple test creation methods and integrating AI-assisted features. The work was performed across two operating systems (macOS and Windows 11) due to platform-specific dependencies.

Katalon Studio was installed on macOS following the Lab 8 instructions (Figures 1–3). A new project was created, and the standard Katalon folder structure (Test Cases, Object Repository, Test Suites, Data Files, Keywords) was initialized. Because Mobile and Desktop testing require Windows-specific drivers and SDKs, a second environment was configured on Windows 11 (see Appendix 4, Table 1). The following sections document each testing domain in detail, beginning with Web/Cloud testing.

PART 2: Web/Cloud Testing

Web testing was performed using the SauceDemo application. All three required test creation methods were implemented. **Record & Playback:** Using the Web Recorder, the login workflow was captured, including entering credentials, clicking the login button, and verifying the landing page. The recorder automatically saved UI elements into the Object Repository (Figures 4–6). Assertions were added to validate successful login (Figure 8). **Manual Mode:** A second test case was created entirely in Manual Mode using built-in keywords such as `openBrowser`, `setText`, `click`, and `verifyElementPresent` (Figure 9). **Script Mode:** The same test was reviewed in Script Mode to examine the underlying Groovy code and confirm that the recorded and manual steps translated correctly into executable script logic (Figure 10).

Object Spy: Additional UI elements were captured using Object Spy to supplement the repository with more stable selectors (Figure 7).

Data-Driven Testing: A CSV file was created (Figure 11), imported (Figure 12), mapped to test variables (Figures 13–14), and executed across five data rows (Figures 15–16). This demonstrated parameterization and multi-iteration execution.

PART 3: API Testing

API testing was performed using Katalon's Sample API Project (Figure 17). Two endpoints were tested: *Create a new user*, and *Find user by ID*. Each request included configured parameters, headers, and verification scripts. The tests were executed individually and then combined into a Test Suite (Figure 19). The suite executed successfully, demonstrating the ability to configure API objects, validate JSON responses, and run grouped tests.

PART 4: Mobile Testing

Research with Claude.ai indicated that Android emulation would be more stable and reliable, so a Windows 11 environment was configured for these tests (see Appendix 4, Table 1). Once the environment was stable, the Katalon Sample Android project was executed. The emulator launched successfully, and the sample tests ran against the API Demos application (Figure 30). Katalon's AI Assistant was used to analyze test steps and provide recommendations (Figures 31–34). The Smoke Test suite passed all tests (Figure 35).

PART 5: Desktop Testing

Desktop testing was the most challenging portion of the lab due to limitations in Katalon Studio 11.2.x and differences between Windows 10 and Windows 11.

Attempt 1: Calculator.exe (UWP Application)

Two major issues occurred:

1. **UWP process handoff** Calculator on Windows 11 is a UWP app. Launching calc.exe hands off to a UWP runtime process, causing FlaUI to lose the window handle. *“Failed to find new window for process ‘calc’ after launching app for 50 seconds.”*
2. **Unstable XPath locators** The Native Windows Recorder generated positional XPath expressions such as: /Window/Window[2]/Custom[1]/Group[1]/Group[5]/Button[10]
These locators were brittle and failed during execution.

These issues are known limitations of Katalon’s FlaUI driver with UWP applications (Katalon, 2026a).

Attempt 2: Notepad.exe (Win32 Application)

To avoid UWP issues, Notepad (notepad.exe) was selected as the Application Under Test.

Configuration Challenges

- Katalon 11.2.1 contains a bug where the Desktop Driver URL does not persist. The FlaUI driver must be started manually before each run (Figures 36–40) (Katalon Community, 2026).
- Katalon does not correctly apply the Windows execution default when creating a Windows project.

Locator Mismatch

The sample project was built for Windows 10, where Notepad’s text area had accessibility ID 15. Windows 11 redesigned Notepad’s UI, and the correct locator is now the element name **“Text Editor.”** This caused the failure: *“No element found with selector ‘accessibility id’ and value ‘15’.*” (See Figure 41.) (Katalon, 2026b). Even though the test did not fully pass, the troubleshooting process demonstrates deep understanding of desktop automation challenges, including OS-level UI changes, driver configuration issues, and locator instability.

PART 6: Analysis & ISP

This lab required applying Input Space Partitioning (ISP) across multiple testing domains.

ISP was most visible in:

- **Web testing**, where login inputs were partitioned into valid/invalid combinations
- **API testing**, where request parameters and response codes formed natural partitions
- **Mobile testing**, where device states (cold start, warm start, emulator configurations) acted as partitions
- **Desktop testing**, where AUT type (UWP vs Win32) created distinct behavioral partitions

Katalon's tooling made ISP straightforward in Web and API contexts but significantly more complex in Desktop testing due to OS-level variability.

The four testing domains also differ in how tractable ISP is to implement, though that may be in some part a function of the framework in use rather than an inherent property of the testing type.

In Katalon Studio, Web and API testing offer well-defined input spaces (form fields, query parameters, HTTP status codes) that map cleanly to partitions and are easily parameterized via Katalon's data-driven features. Mobile testing introduces environmental partitions (emulator state, OS version, device configuration) that are harder to enumerate in advance but still manageable within a controlled test environment.

Desktop testing proved the most difficult to test: the AUT type (UWP vs. Win32) functioned as an implicit partition that was only discovered through failure, highlighting a gap between ISP theory and the reality of desktop automation, where the input space includes the OS itself. A different framework with more mature UWP support or built-in environment abstraction might surface these partitions earlier in the test design process rather than at execution time.

PART 7: AI Features & Conclusion

Katalon Studio includes an AI Assistant intended to support test creation, troubleshooting, and analysis. In practice, the AI experience varied significantly depending on the operating system. During this lab, the AI features were fully available only on Windows. On macOS, the AI panel did not appear, and none of the AI-assisted capabilities—such as step analysis, locator suggestions, or automated troubleshooting—were accessible. This created a noticeable difference in workflow between the two environments.

On Windows, the AI Assistant was integrated directly into the interface and could analyze recently executed test steps, highlight failures, and provide explanations for common issues. It was particularly helpful during Mobile testing, where it offered clarifications about Appium behavior and element identification. The assistant also provided contextual summaries of test results, which made it easier to understand the sequence of actions executed during a test run.

However, the AI Assistant had clear limitations. Most notably, it did not appear to have access to Katalon's own discussion boards, community posts, or known-issues database. As a result, the AI could describe an error message but could not identify when the underlying cause was a documented Katalon bug. For example, the Desktop Driver URL persistence issue in Katalon 11.2.1 is widely discussed in the community, but the AI Assistant did not recognize it as a known defect (Katalon Community, 2026). Similarly, the Windows 10 vs. Windows 11 Notepad locator mismatch is a common problem, yet the AI Assistant treated it as a generic locator failure rather than a version-specific UI change. This gap highlights an opportunity for improvement: integrating community knowledge and known-issue tracking into the AI's reasoning would make it far more effective.

Due to confirmed bugs in Katalon Studio 11.2.1 that prevented successful Desktop test execution, two Python-based alternatives were identified as viable replacements for the Windows 11 environment. These alternatives leverage tools already present in the lab environment (Appium, Python) and target the same application under test (Notepad, a classic Win32 app), enabling a meaningful methodological comparison despite the Katalon failure.

Using two computers added complexity to the workflow. While macOS was convenient for Web and API testing, iOS automation is significantly more restricted and would have required multiple developer accounts, provisioning profiles, and additional approvals. Conversations with Copilot and Claude confirmed that iOS testing would introduce unnecessary overhead for this assignment. Since both laptops were available, using Windows for Mobile and Desktop testing provided a more realistic and complete view of Katalon's capabilities across platforms. It also made the differences in AI functionality more apparent.

Overall, the AI tools were helpful for explaining errors, suggesting locator strategies, and reviewing test logic, but they were not able to overcome platform-level bugs or configuration issues. The experience demonstrated both the value and the current limitations of AI-assisted testing. As Katalon continues to evolve, deeper integration with community knowledge, known-issue tracking, and cross-platform parity would significantly enhance the usefulness of its AI features.

In summary, this lab provided hands-on experience across multiple testing domains and highlighted both the strengths and limitations of Katalon Studio. The combination of cross-platform testing, ISP analysis, and AI-assisted troubleshooting offered a comprehensive view of modern test automation practices.

APPENDIX 1: Tool Installation & Project Setup on Mac

Figure 1. Katalon Installation

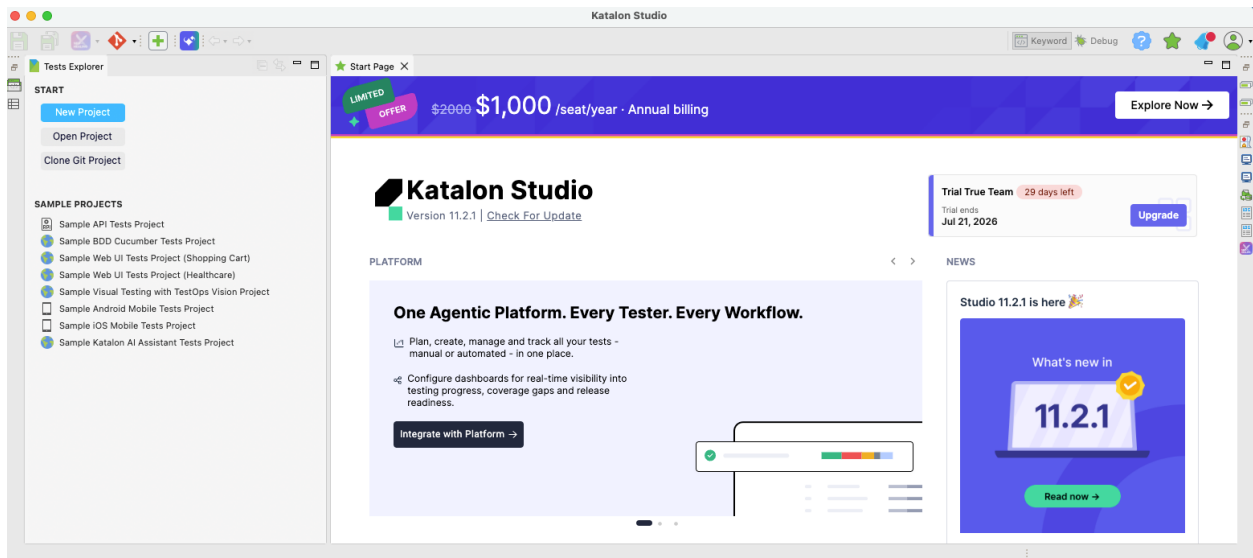


Figure 2. Confirmation of Katalon Recorder installation

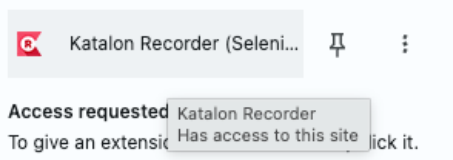
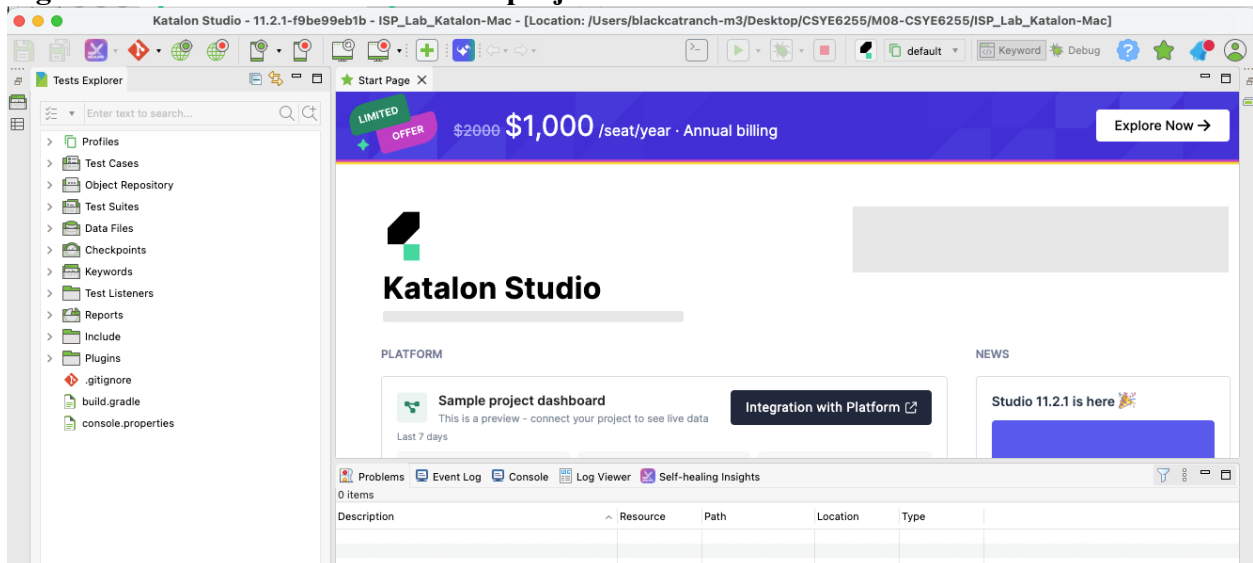


Figure 3. Creation of a new Katalon project



APPENDIX 2: Cloud/Web Project Setup & Execution on Mac

Figure 4. Creating a new Test Case container

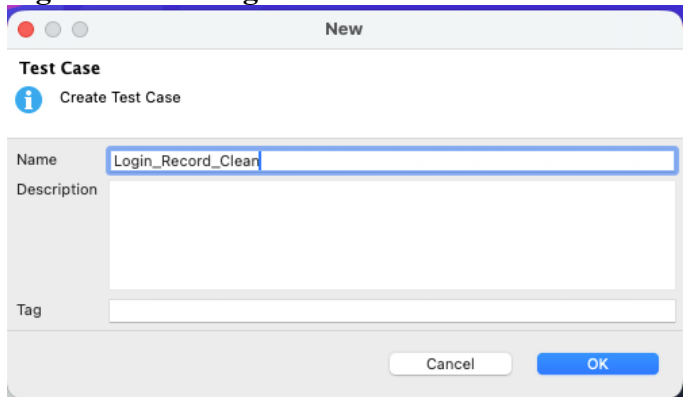


Figure 5. Web Record & Save to Object Repo

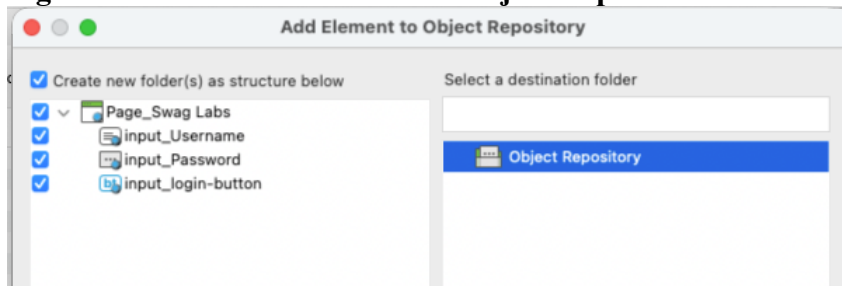


Figure 6. Login_Record_Clean Object Repo & Play Confirmation

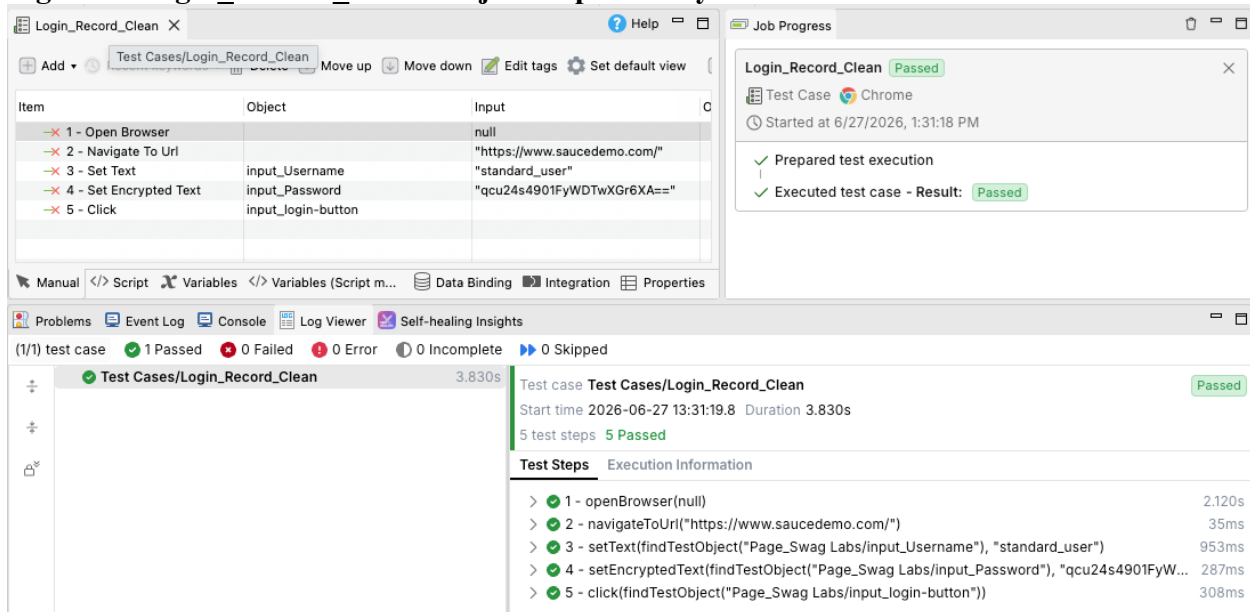


Figure 7. Capture Objects with Object Spy

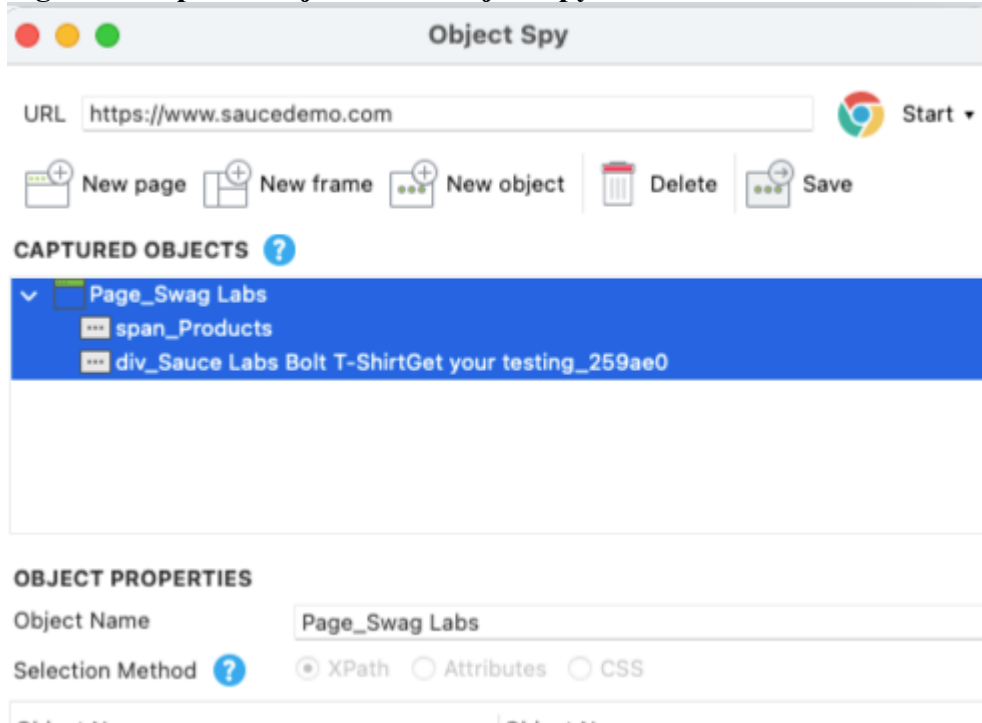


Figure 8. Confirm Addition of Assertions to Test & Successful Test

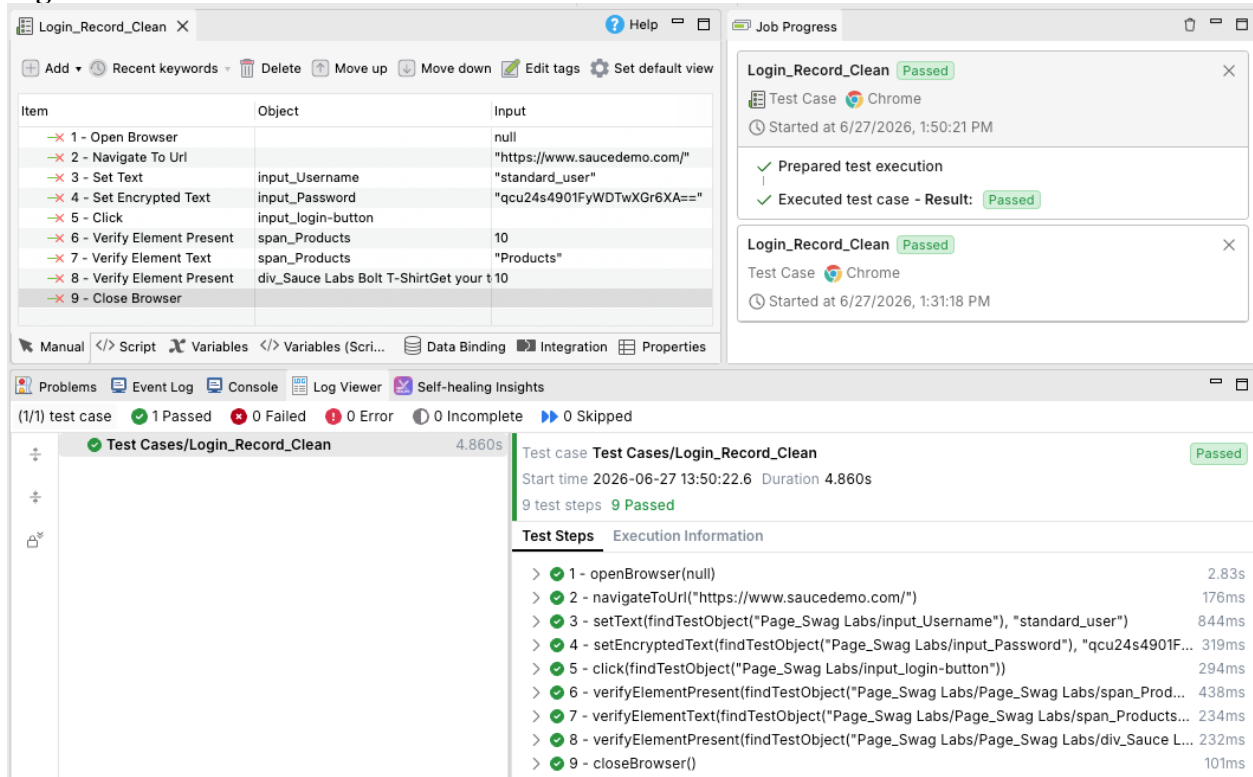


Figure 9. Manual Mode Test Creation & Successful Execution

The screenshot displays the Katalon Studio Manual Mode Test Creation & Successful Execution interface. The main window shows a test case named "Test_user" with 8 steps: 1 - Open Browser, 2 - Navigate To Url, 3 - Set Text, 4 - Set Text, 5 - Click, 6 - Verify Element Present, 7 - Verify Element Text, and 8 - Close Browser. The test case is marked as "Passed". The Job Progress panel on the right shows the test case execution details, including the start time (2026-06-27 14:25:01 PM) and the result (Passed). The bottom status bar indicates that the test case was executed successfully with 1 Passed, 0 Failed, 0 Error, 0 Incomplete, and 0 Skipped.

Item	Object	Input
1 - Open Browser		""
2 - Navigate To Url		"https://www.saucedemo.com"
3 - Set Text	input_username	"standard_user"
4 - Set Text	input_password	"secret_sauce"
5 - Click	input_login-button	
6 - Verify Element Present	span_Products	10
7 - Verify Element Text	span_Products	"Products"
8 - Close Browser		

Test Steps Execution Information

Test Steps	Execution Information
1 - openBrowser("")	1.895s
2 - navigateToUrl("https://www.saucedemo.com")	177ms
3 - setText(findTestObject("Page_Swag Labs/input_username"), "standard_user")	851ms
4 - setText(findTestObject("Page_Swag Labs/input_password"), "secret_sauce")	527ms
5 - click(findTestObject("Page_Swag Labs/input_login-button"))	314ms
6 - verifyElementPresent(findTestObject("Page_Swag Labs/Page_Swag Labs/span_Products"), 10)	449ms
7 - verifyElementText(findTestObject("Page_Swag Labs/Page_Swag Labs/span_Products"), "Products")	250ms
8 - closeBrowser()	96ms

Figure 10. Script Mode Mode Test View

The screenshot displays the Katalon Studio Script Mode Test View interface. The main window shows the script code for the test case "Test_user". The script is written in Java and uses the Katalon API to perform the test steps. The script is as follows:

```

1 import static com.kms.katalon.core.checkpoint.CheckpointFactory.findCheckpoint
2 import static com.kms.katalon.core.testcase.TestCaseFactory.findTestCase
3 import static com.kms.katalon.core.testdata.TestDataFactory.findTestData
4 import static com.kms.katalon.core.testobject.ObjectRepository.findTestObject
5 import static com.kms.katalon.core.testobject.ObjectRepository.findWindowsObject
6 import com.kms.katalon.core.checkpoint.Checkpoint as Checkpoint
7 import com.kms.katalon.core.cucumber.keyword.CucumberBuiltinKeywords as CucumberKW
8 import com.kms.katalon.core.mobile.keyword.MobileBuiltinKeywords as Mobile
9 import com.kms.katalon.core.model.FailureHandling as FailureHandling
10 import com.kms.katalon.core.testcase.TestCase as TestCase
11 import com.kms.katalon.core.testdata.TestData as TestData
12 import com.kms.katalon.core.testng.keyword.TestNGBuiltinKeywords as TestNGKW
13 import com.kms.katalon.core.testobject.TestObject as TestObject
14 import com.kms.katalon.core.webservice.keyword.WSBuiltInKeywords as WS
15 import com.kms.katalon.core.webui.keyword.WebUiBuiltInKeywords as WebUI
16 import com.kms.katalon.core.windows.keyword.WindowsBuiltinKeywords as Windows
17 import internal.GlobalVariable as GlobalVariable
18 import org.openqa.selenium.Keys as Keys

19
20 WebUI.openBrowser("")
21
22 WebUI.navigateToUrl('https://www.saucedemo.com')
23
24 WebUI.setText(findTestObject('Page_Swag Labs/input_username'), 'standard_user')
25
26 WebUI.setText(findTestObject('Page_Swag Labs/input_password'), 'secret_sauce')
27
28 WebUI.click(findTestObject('Page_Swag Labs/input_login-button'))
29
30 WebUI.verifyElementPresent(findTestObject('Page_Swag Labs/Page_Swag Labs/span_Products'), 10)
31
32 WebUI.verifyElementText(findTestObject('Page_Swag Labs/Page_Swag Labs/span_Products'), 'Products')
33
34 WebUI.closeBrowser()
35

```

Figure 11. Create CSV file

```
loginData.csv X
Users > blackcatranch-m3 > Desktop > CSYE6255 > M08-CSYE6255 > loginData.csv
1 username,password
2 standard_user,secret_sauce
3 problem_user,secret_sauce
4 performance_glitch_user,secret_sauce
5 error_user,secret_sauce
6 visual_user,secret_sauce
```

Figure 12. Confirm Imported CSV Data

Login_Record_Clean Test_user *New Test Data X Help

File's Information

File Name /Users/blackcat Browse... Separator COMMA

☒ Use first row as header ☐ Use relative path

No.	username	password
1	standard_user	secret_sauce
2	problem_user	secret_sauce
3	performance_glitch_user	secret_sauce
4	error_user	secret_sauce
5	visual_user	secret_sauce

Figure 13. Creating Test Variables

*Login_Record_Clean X Test_user *New Test Data Help

Add Recent keywords Delete Move up Move down Edit tags Set default view

	Object	Input
1 - Open Browser		null
2 - Navigate To Url		"https://www.saucedemo.com/"
3 - Set Text	input_username	"username"
4 - Set Text	input_password	"password"
5 - Click	input_login-button	
6 - Verify Element Present	span_Products	10
7 - Verify Element Text	span_Products	"Products"
8 - Verify Element Present	div_Sauce Labs Bolt T-ShirtGet your t	10
9 - Close Browser		

Figure 14. Mapping Test Variables

Variable Binding

Set Type Set Test Data Set Test Name password

No.	Name	Default value	Type	Test Data	Value
1	username	"	Data Column	1 - Data Files/New	username
2	password	"	Data Column	1 - Data Files/New	password

Figure 15. Update Test Data Type to “Many”

Test Data

⊕ Add ⊖ Delete ⬆ Move Up ⬇ Move Down ⌘ Map All

No.	ID	Data Iteration	Type
1	Data Files/New Test Data	All	Many

Variable Binding

📄 Set Type 📄 Set Test Data 📄 Set Test Name password ⓘ ⓘ

No.	Name	Default value	Type	Test Data	Value
1	username	"	Data Column	1 - Data Files/New	username
2	password	"	Data Column	1 - Data Files/New	password

Figure 16. Confirmation of Successful Test with CSV imported data for 5 cases

🔍 Problems 📄 Event Log 📄 Console 📄 Log Viewer 🧠 Self-healing Insights

(5/5) test cases 5 Passed 0 Failed 0 Error 0 Incomplete 0 Skipped

Test Suites/Login_testSuite 25.333s

> Test Cases/Login_Record_Clean 23.983s

Test case **Test Cases/Login_Record_Clean** Passed

Start time 2026-06-27 15:23:53.6 Duration 5.19s

9 test steps 9 Passed

Test Steps Data Binding

- > 1 - openBrowser(null) 1.977s
- > 2 - navigateToUrl("https://www.saucedemo.com/") 197ms
- > 3 - setText(findTestObject("Page_Swag Labs/input_Username"), username) 857ms
- > 4 - setText(findTestObject("Page_Swag Labs/input_Password"), password) 527ms
- > 5 - click(findTestObject("Page_Swag Labs/input_login-button")) 306ms
- > 6 - verifyElementPresent(findTestObject("Page_Swag Labs/Page_Swag Labs/span_... 456ms
- > 7 - verifyElementText(findTestObject("Page_Swag Labs/Page_Swag Labs/span_Prod... 239ms
- > 8 - verifyElementPresent(findTestObject("Page_Swag Labs/Page_Swag Labs/div_Sa... 237ms
- > 9 - closeBrowser() 101ms

APPENDIX 3: API Testing

Figure 17. API Testing – New Sample Project (Lab8_API_Demo_Project-2) & Test “Create a new user”

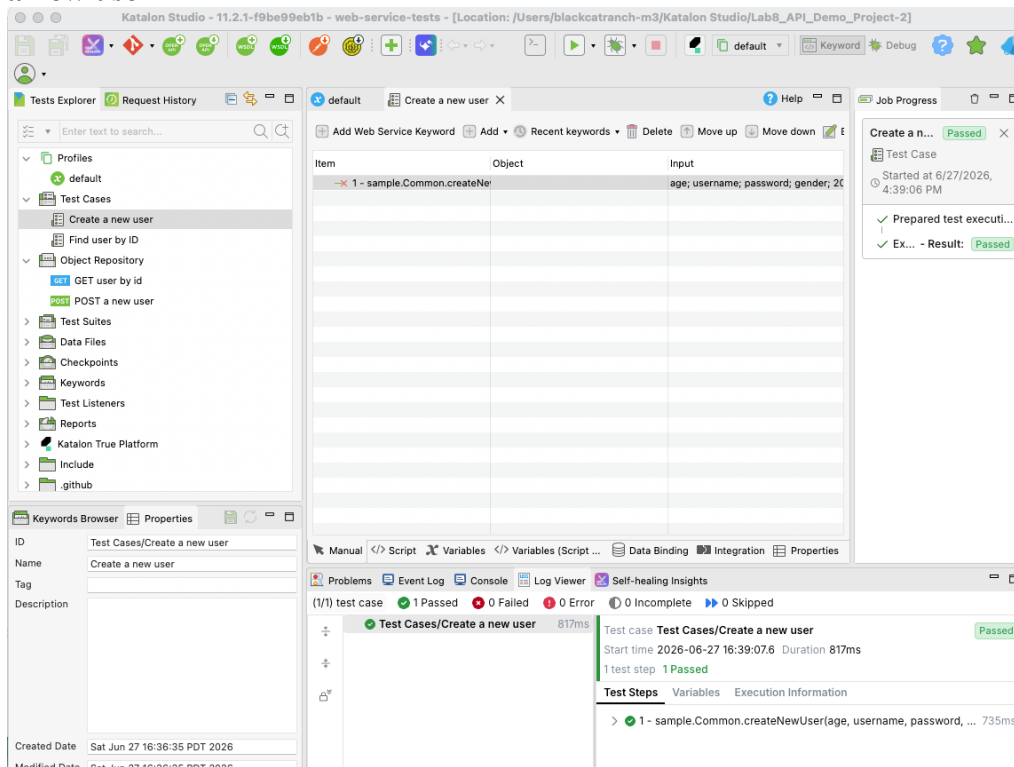


Figure 18. API Testing – Test “Find user by ID”

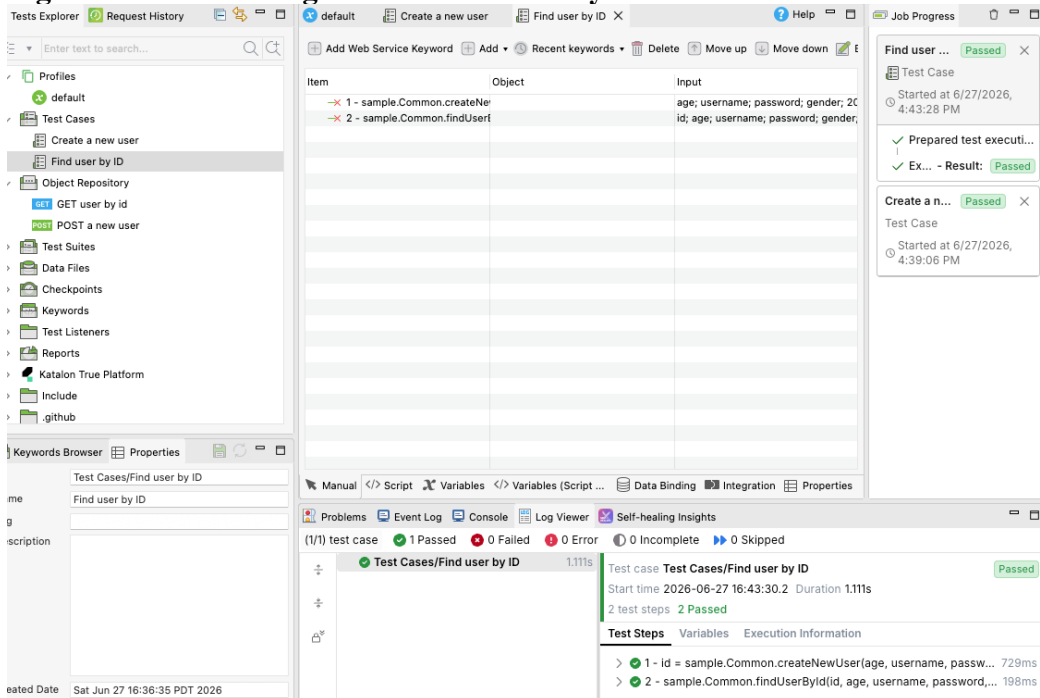


Figure 19. API Testing – Run Test Suite

Katalon Studio - 11.2.1-f9be99eb1b - web-service-tests - [Location: /Users/blackcatranch-m3/Katalon Studio/Lab8_API_Demo_Project-2]

Tests Explo... Request Hi... default Create a new user Find user by ID web-service-tests - All Test Cases X Job Progress

Enter text to search... Profiles default Test Cases Create a new user Find user by ID Object Repository GET user by id POST POST a new user Test Suites web-service-tests - All Test Case Data Files Checkpoints Keywords

Execution Information

Add Delete Move Up Move Down View Test Run History Show Data Binding

Enter text to search...

No.	ID	Description	Flakiness (%)	Latest Run	Avg. Duration
1	../Create a new user		--	--	--
2	../Find user by ID		--	--	--

Main </> Script Integration Result Analytics Problems Event Log Console Log Viewer Self-healing Insights

(2/2) test cases 2 Passed 0 Failed 0 Error 0 Incomplete 0 Skipped

Test Suites/web-service-tests - All Test Cases 2.598s
sampleBeforeTestSuite 23ms
Test Cases/Create a new user 755ms
Test Cases/Find user by ID 451ms

Test case Test Cases/Create a new user
Start time 2026-06-27 16:45:27.3 Duration 755ms
1 test step 1 Passed

Test Steps Variables
1 - sample.Common.createNewUser(age, username, password, gender, 200) 711ms

web-service-tests - All Test Cases Passed
Test Suite
Started at 6/27/2026, 4:45:24 PM
Prepared test execution
Executed 2/2 test case(s) - Result: 2 Passed
Generated and uploaded test reports

Find user by ID Passed
Test Case
Started at 6/27/2026, 4:43:28 PM

Create a new user Passed
Test Case
Started at 6/27/2026, 4:39:06 PM

Keywords Bro... Properties

ID Test Suites/web-service-tests
Name web-service-tests - All Test Ca
Tag
Description

Created Date Sat Jun 27 16:36:35 PDT 2026
Modified Date Sat Jun 27 16:36:35 PDT 2026

APPENDIX 4: Android Testing

Table 1. Windows 11 Environment Setup:

Windows Application Driver	Figure 20
Katalon Studio mobile configuration	Figure 21
Android Studio	Figure 22
Android SDK	Figure 23
Android Emulator	Figures 24–25
Node.js	Figure 26
Appium	Figures 27–29

Figure 20. Android Testing – setup Windows Application Driver

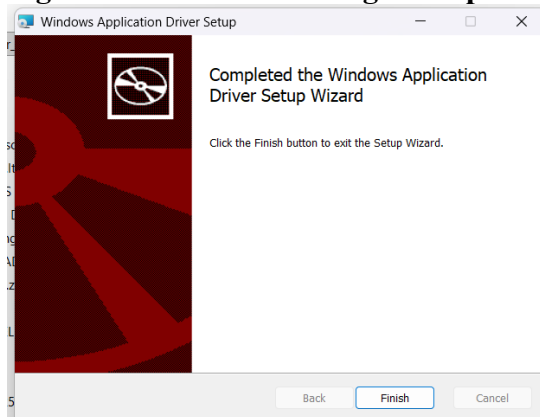


Figure 21. Android Testing – setup Katalon Studio

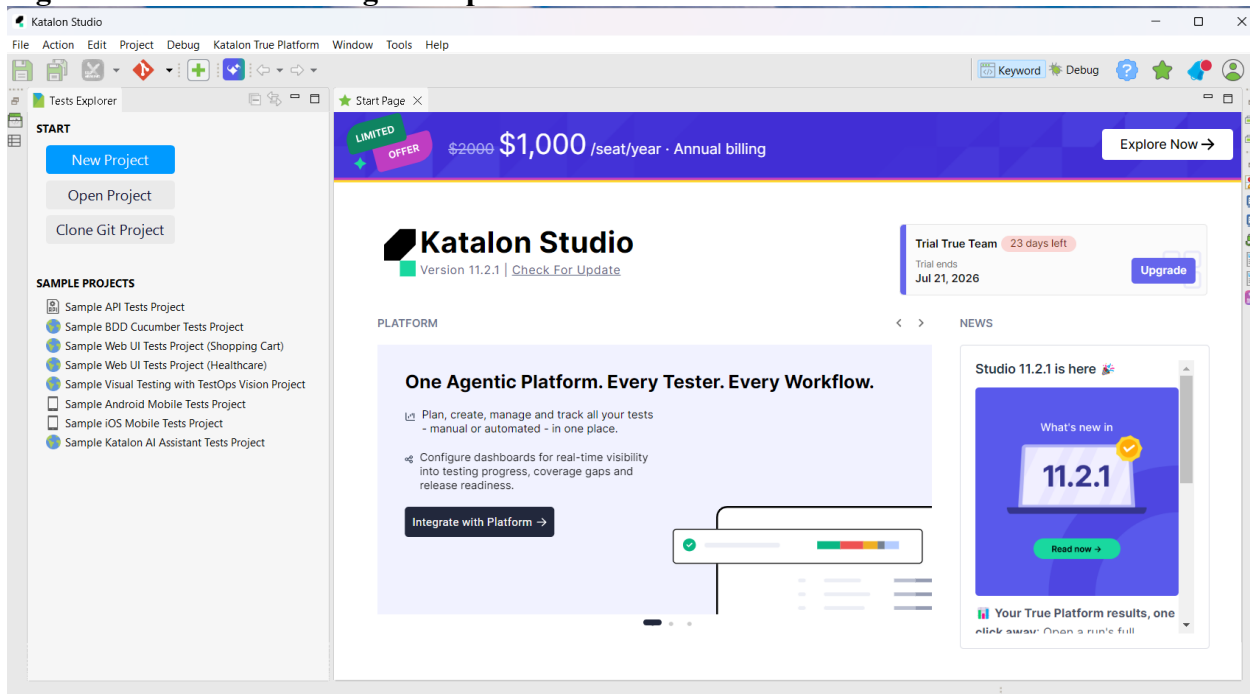


Figure 22. Android Testing – setup Android Studio

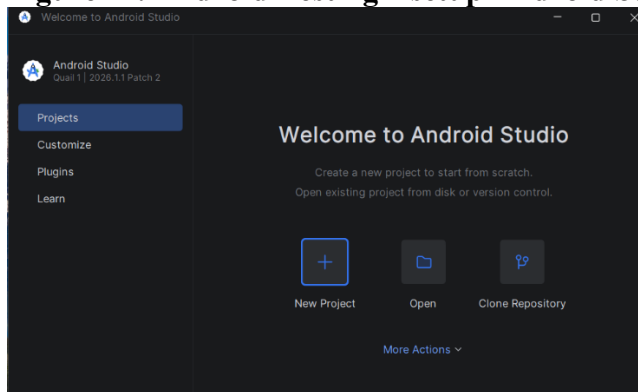


Figure 23. Android Testing – setup Android SDK

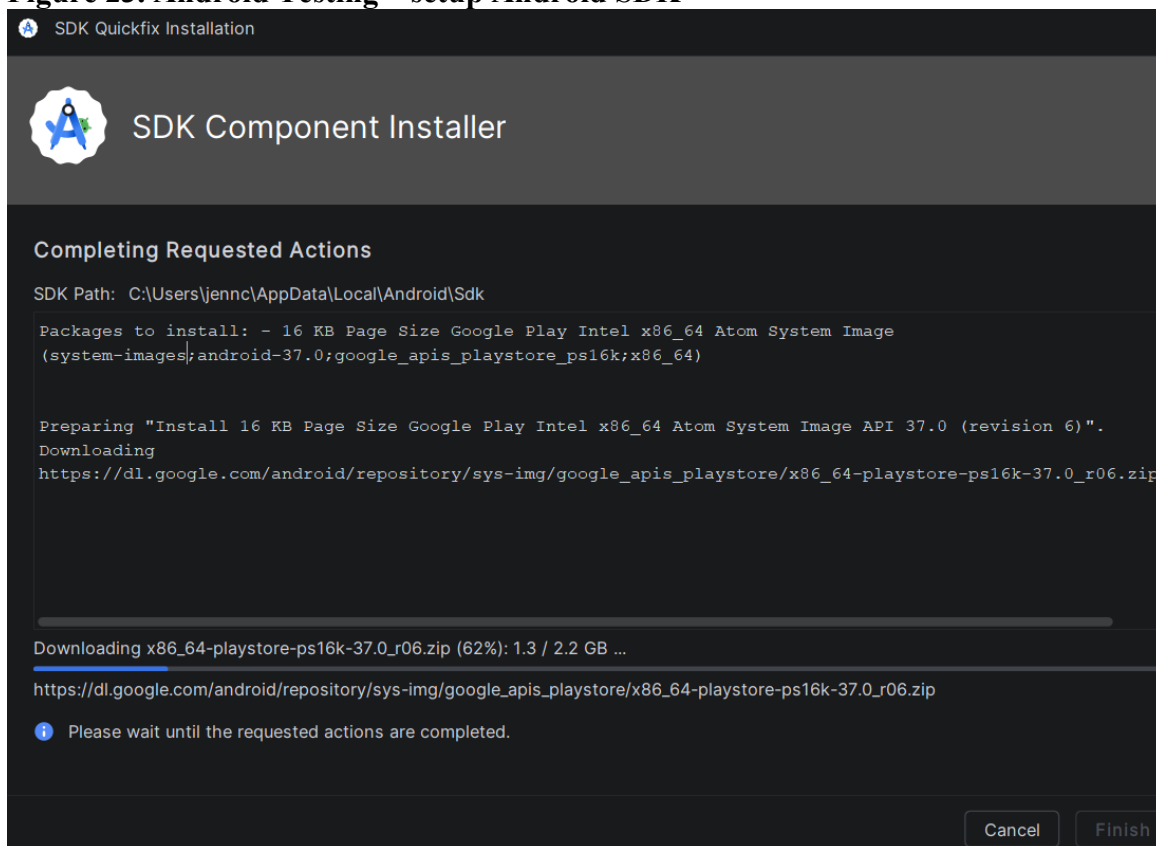


Figure 24. Android Testing – setup Android emulator

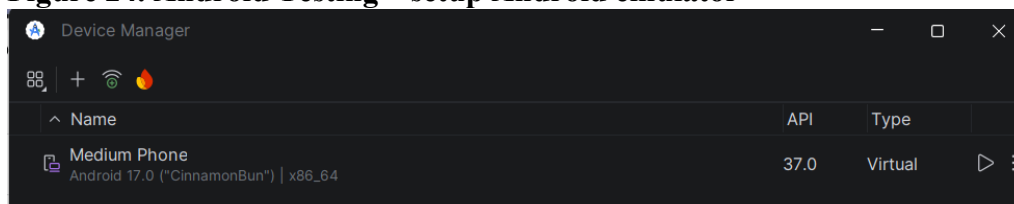


Figure 25. Android Testing – running Android emulator

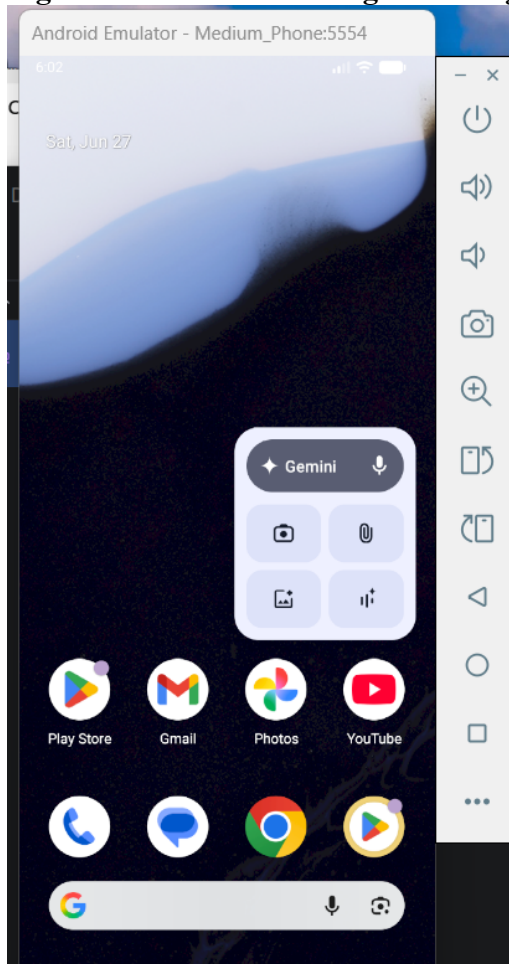
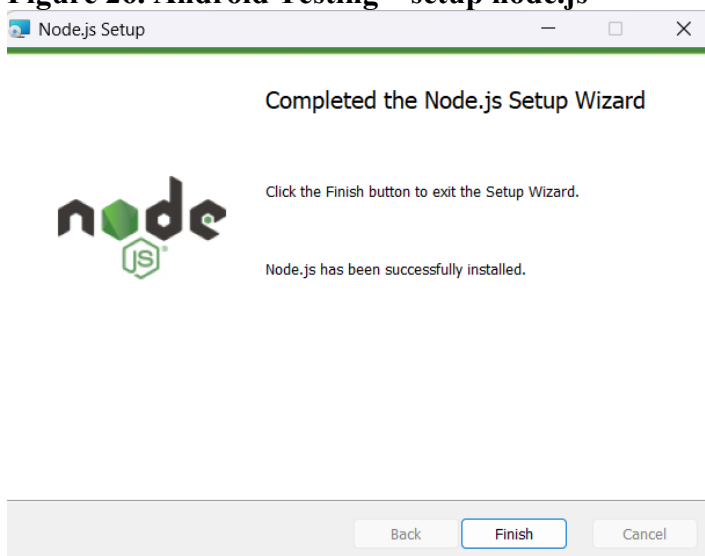


Figure 26. Android Testing – setup node.js



Figures 27-29. Android Testing – setup Appium

```
PS C:\Users\jennnc> npm install -g appium
added 293 packages in 1m

65 packages are looking for funding
  run 'npm fund' for details
npm warn allow-scripts 1 package has install scripts not yet covered by allowScripts:
npm warn allow-scripts   appium@3.5.2 (postinstall: node ./scripts/autoinstall-extensions.js)
npm warn allow-scripts
npm warn allow-scripts Run 'npm approve-scripts --allow-scripts-pending' to review, or 'npm approve-scripts <pkg>' to allow.
npm notice
npm notice New minor version of npm available! 11.16.0 -> 11.17.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.17.0
npm notice To update run: npm install -g npm@11.17.0
npm notice
PS C:\Users\jennnc>
```

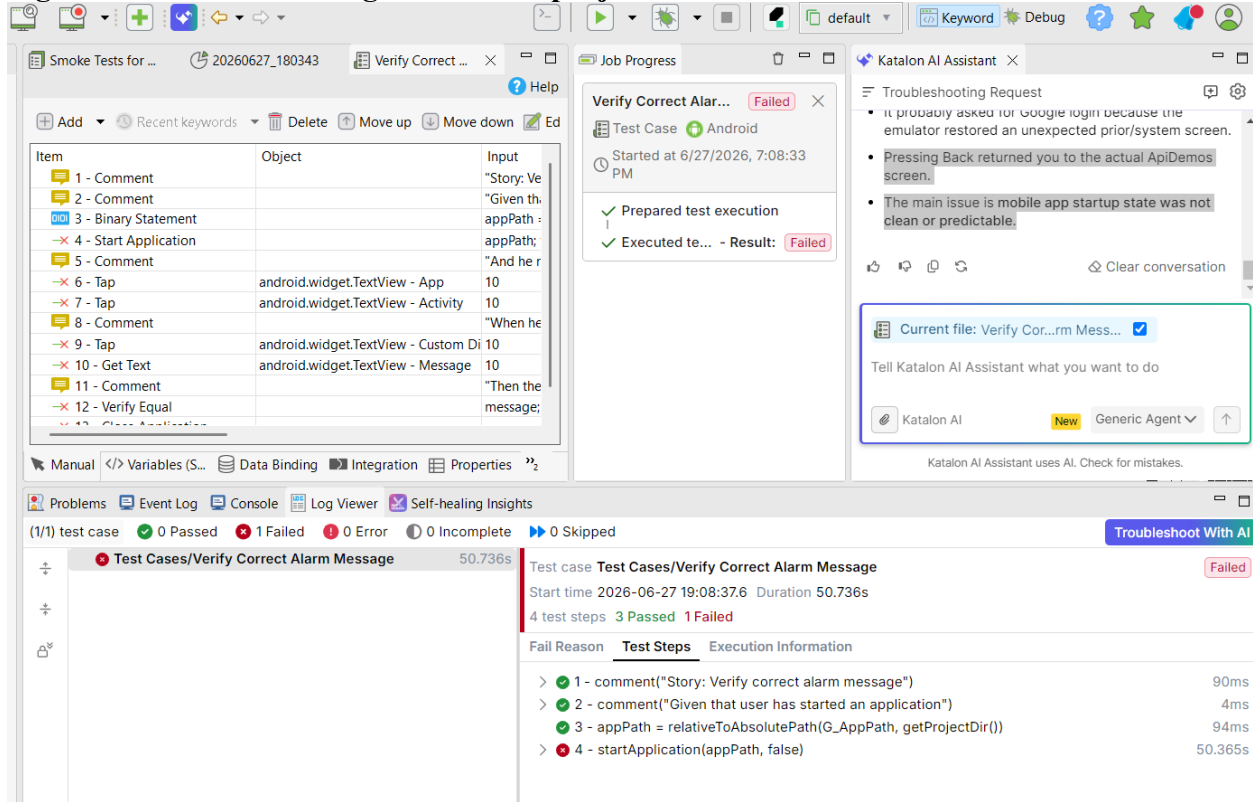
```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

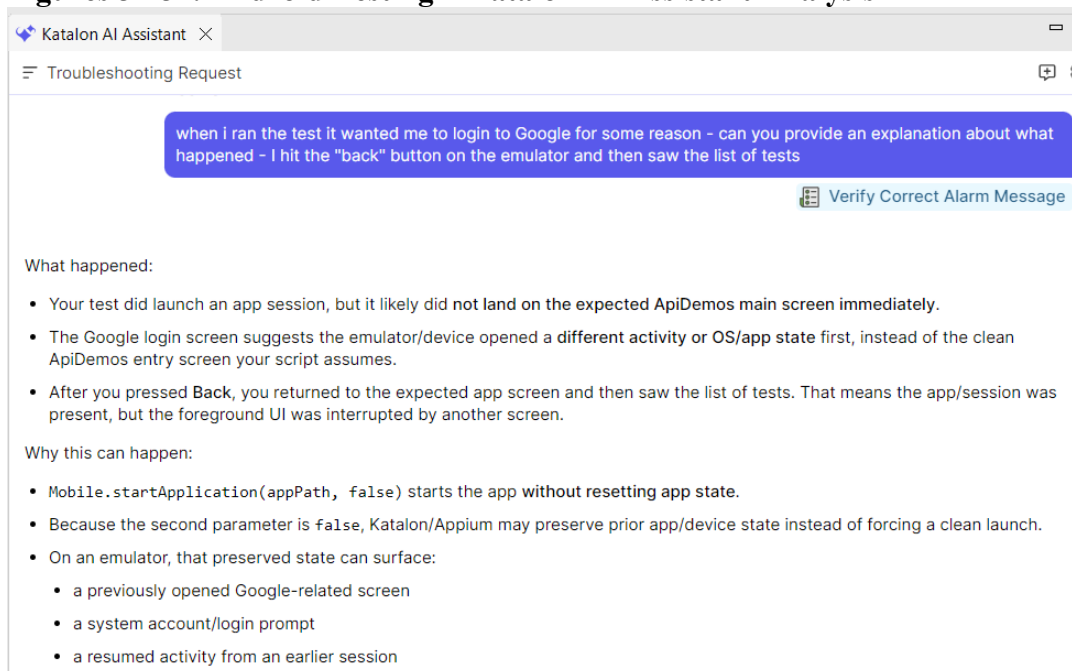
PS C:\Users\jennnc> appium driver install uiautomator2
'Checking if 'appium-uiautomator2-driver' is compatible(node:28292) [DEP0190] DeprecationWarning: Passing args to a child process with shell option true can lead to security vulnerabilities, as the arguments are not escaped, only concatenated.
(Use 'node --trace-deprecation ...' to show where the warning was created)
✓Checking if 'appium-uiautomator2-driver' is compatible
✓Installing 'uiautomator2'
✓Validating 'uiautomator2'
[Driver uiautomator2@8.0.0 successfully installed
- automationName: UiAutomator2
- platformNames: ["Android"]
PS C:\Users\jennnc>
```

```
Windows PowerShell
http://127.0.0.1:4723/ (only accessible from the same host)
[Appium] No drivers have been installed in C:\Users\jennnc\.appium. Use the "appium driver" command to install the one(s) you want to use.
[Appium] No plugins have been installed. Use the "appium plugin" command to install the one(s) you want to use.
[Appium] Received SIGINT - shutting down
[AppiumDriver@c810] There are no active sessions for cleanup
[HTTP] Closing Appium HTTP server
[HTTP] Appium HTTP server has been successfully closed after 0ms
PS C:\Users\jennnc> appium
[Appium] Welcome to Appium v3.5.2
[Appium] The autodetected Appium home path: C:\Users\jennnc\.appium
[Appium] Attempting to load driver uiautomator2...
[Appium] Requiring driver at C:\Users\jennnc\.appium\node_modules\appium-uiautomator2-driver\build\lib\index.js
[Appium] AndroidUiautomator2Driver has been successfully loaded in 0.536s
[Appium] Appium REST http interface listener started on http://0.0.0.0:4723
[Appium] You can provide the following URLs in your client code to connect to this server:
    http://192.168.56.1:4723/
    http://10.0.0.1:4723/
    http://192.168.1.20:4723/
    http://127.0.0.1:4723/ (only accessible from the same host)
[Appium] Available drivers:
[Appium]   - uiautomator2@8.0.0 (automationName 'UiAutomator2')
[Appium] No plugins have been installed. Use the "appium plugin" command to install the one(s) you want to use.
```

Figure 30. Android Testing – run demo projects



Figures 31-34. Android Testing – Katalon AI Assistant Analysis



Katalon AI Assistant X

Troubleshooting Request

- a resumed activity from an earlier session
- a first-run/setup flow from Google Play services or the emulator image

Why pressing Back helped:

- The Back button dismissed that unexpected foreground screen.
- Once dismissed, the emulator showed the actual ApiDemos screen your test expected.
- So the issue was likely **environment/app state interference**, not the test steps themselves.

Most probable root cause:

- The test assumes this line always lands on the ApiDemos home screen:

```
groovy
Mobile.startApplication(appPath, false)
```

- But with `false`, the app can resume into whatever state the emulator had cached.

What this means for your test:

- The script is brittle at startup.
- It does not verify the initial screen before tapping:
 - 'Application/android.widget.TextView - App'
- If a Google sign-in or some other overlay appears, the test can fail even though the app itself is installed and runnable.

Katalon AI Assistant X

Troubleshooting Request

Best explanation in plain terms:

- Katalon/Appium opened the emulator app session.
- The emulator restored or showed an unexpected Google-related screen.
- You manually cleared that screen with Back.
- After that, the app displayed the expected ApiDemos list, which is where the test intended to begin.

How to reduce this next time:

- Use a clean app launch if appropriate:

```
groovy
Mobile.startApplication(appPath, true)
```

This resets app state more aggressively.

- Add an initial verification/wait step to confirm the app is really on the expected home screen before tapping.
- Consider closing unexpected popups or checking for alternate startup screens.
- Use a cleaner emulator/device state before execution.

Example safer startup pattern:

```
groovy
```

```
def appPath = PathUtil.relativeToAbsolutePath(GlobalVariable.G_AppPath, RunConfiguration.getProjectDir())
Mobile.startApplication(appPath, true)
Mobile.waitForElementPresent(findTestObject('Application/android.widget.TextView - App'), 20)
Mobile.tap(findTestObject('Application/android.widget.TextView - App'), 10)
```

So in short:

- It probably asked for Google login because the emulator restored an unexpected prior/system screen.
- Pressing Back returned you to the actual ApiDemos screen.
- The main issue is **mobile app startup state was not clean or predictable**.

Figure 35. Android Testing – Smoke Tests for API Demos App suite (all passed)

Execution Information

Enter text to search...

No.	ID	Description	Flakiness (%)	Latest Run	Avg
1	../Verify Correct Alarm Message	An automation s	--	--	--
2	../Verify Last Items In List	An automation s	--	--	--

Job Progress

Smoke Tests for API Demos App **Passed**

Test Suite **Android**

Started at 6/27/2026, 7:24:19 PM

- ✓ Prepared test execution
- ✓ Executed 2/2 test case(s) - Result: **2 Passed**
- ✓ Generated and uploaded test reports

Verify Correct Alarm Message **Passed**

Test Case **Android**

Started at 6/27/2026, 7:22:32 PM

Verify Correct Alarm Message **Failed**

Test Case **Android**

Started at 6/27/2026, 7:08:33 PM

Test Results Summary

(2/2) test cases **2 Passed** **0 Failed** **0 Error** **0 Incomplete** **0 Skipped**

Test Suites/Smoke Tests for API Demos App 44.365s

- ✓ **sampleBeforeTestSuite** 146ms
- ✓ Test Cases/Verify Correct Alarm Message 24.335s
- ✓ Test Cases/Verify Last Items In List 16.255s

Test step sampleBeforeTestSuite

Start time 2026-06-27 19:24:24.4 Duration 146ms

Test Steps

- ✓ 1 - KatalonHelper.updateInfo() 123ms

APPENDIX 5: Desktop Testing with Calculator.exe

Summary authored by Claude.ai:

During Desktop testing, Windows Calculator (calc.exe) was initially selected as the Application Under Test. Two issues were encountered.

First, Calculator on Windows 11 is packaged as a Universal Windows Platform (UWP) application. When launched via the traditional executable path `C:\Windows\System32\calc.exe`, the process immediately hands off to a UWP runtime process. The FlaUI driver loses track of the new process window, resulting in the error: *"Failed to find new window for process 'calc' after launching app for 50 seconds."*

Second, the Native Windows Recorder captured button locators using positional XPath expressions such as `/Window/Window[2]/Custom[1]/Group[1]/Group[5]/Button[10]`. These locators are session-specific and do not reliably resolve when the test is executed through the FlaUI driver, resulting in `NoSuchElementException` timeouts.

Both issues reflect known limitations of Katalon Studio 11.2.1's FlaUI-based desktop driver with UWP applications. Notepad (notepad.exe) was selected as an alternative AUT because it is a stable Win32 application with predictable element locators.

Figure 36. Desktop Testing – Enabling Desktop Driver

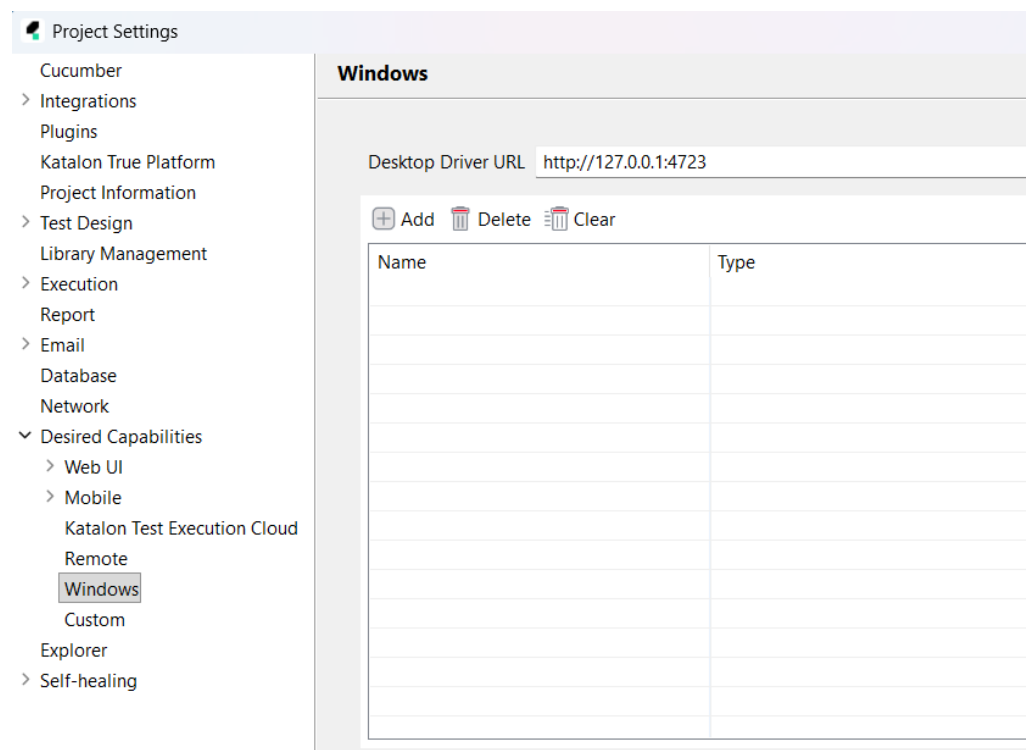


Figure 37. Desktop Testing – Using Katalon AI to troubleshoot again

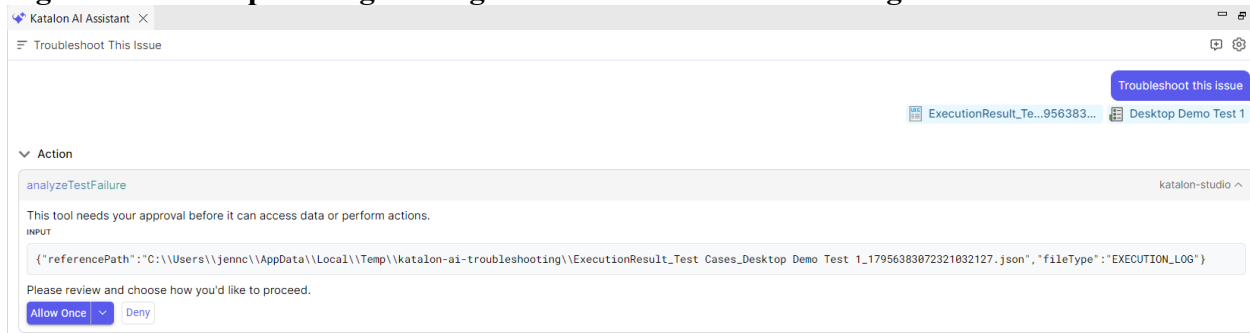


Figure 38. Desktop Testing – Updating the App Title

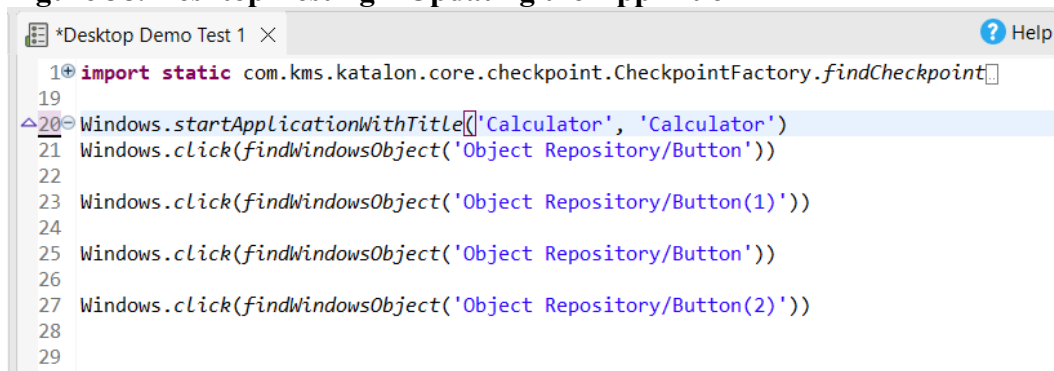
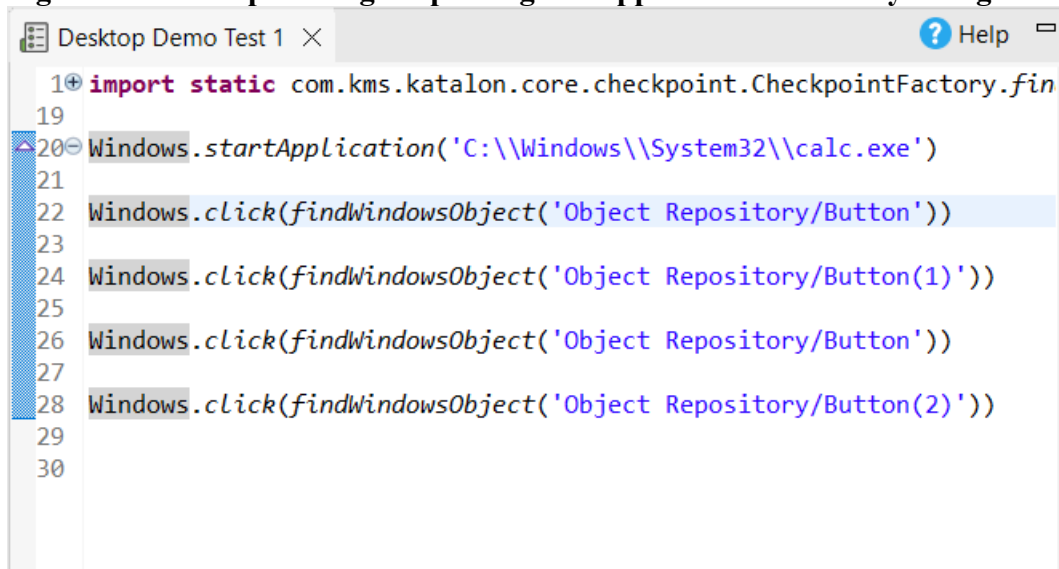


Figure 39. Desktop Testing – Updating the App Path with Groovy string literal



APPENDIX 6: Desktop Testing with Notepad.exe (sample-windows-application-test)

Summary created by Claude.ai:

Following the Windows Calculator UWP issues documented above, Notepad (C:\Windows\System32\notepad.exe) was selected as an alternative Application Under Test for desktop testing. Notepad is a Classic Win32 application, which avoids the UWP process handoff problem that caused Calculator to fail.

Environment configuration required

Katalon Studio 11.2.1 has a confirmed bug where the Desktop Driver URL field in Project Settings → Desired Capabilities → Windows does not persist between sessions. The FlaUI driver must be started manually before each test execution using the following PowerShell command:

```
cd "C:\Users\[user]\.katalon\packages\KS-
```

```
11.2.1\configuration\resources\extensions\FlaUIWebDriver"
```

```
.\FlaUI.WebDriver.exe --urls=http://127.0.0.1:4723 --environment=Production
```

Additionally, Project Settings → Execution → Default execution must be manually set to Windows rather than Chrome, as Katalon 11.2.1 does not correctly apply this setting when a Windows project is created.

Test execution result

Using Katalon's official sample Windows test project (github.com/katalon-studio/sample-windows-application-test), the SampleNotepad test case was executed. Step 1 — startApplication("C:\Windows\System32\notepad.exe") — passed successfully, confirming that the FlaUI driver connected correctly and launched Notepad.

Step 2 — setText(findWindowsObject("Object Repository/Edit"), "Welcome to Katalon Studio") — failed with the error: No element found with selector 'accessibility id' and value '15'. The sample project was built for Windows 10, where Notepad's text area had accessibility ID 15. In Windows 11, Microsoft redesigned Notepad as part of a broader UI refresh, and the accessibility ID for the text editor area changed. The correct locator strategy for Windows 11 Notepad is name with value Text Editor.

Summary

Desktop testing in Katalon Studio 11.2.x involves multiple platform-specific configuration steps not required for Web, API, or Mobile testing. The combination of the Desktop Driver URL persistence bug, the Windows execution default not applying on project creation, and locator incompatibilities between the Windows 10 sample project and Windows 11 reflect real-world challenges in desktop test automation: tool version mismatches, OS-level UI changes, and driver configuration complexity. These are not unique to Katalon — they are characteristic of desktop automation as a discipline, where UI element identification is tightly coupled to OS version and application release cycle.

Figure 40. Adding Desktop Driver URL into the settings

Windows

Desktop Driver URL

Add Delete Clear

Name	Type	Value
desiredCapabilities	Dictionary	{}
winAppDriverUrl	String	http://127.0.0.1:4723

Figure 41. Desktop Testing with Notepad – Multiple Failures

SampleNotepad x Edit

Add Recent keywords Delete Move up Move down Edit tags

Item	Object	Input
1 - Start Application		"C:\Windows\System
2 - Set Text	Edit	"Welcome to Katalon
3 - Method Call Statement		Thread.sleep(GlobalV
4 - Click	CloseButton	
5 - Method Call Statement		Thread.sleep(GlobalV
6 - Click	DontSaveButton	
7 - Method Call Statement		Windows.getDriver()

Manual </> Variables (S... Data Binding Integration Properties

Job Progress

SampleNotepad **Failed**

Test Case Windows
Started at 6/27/2026, 8:55:50 PM
Prepared test execution
Executed test case - Result: **Failed**

SampleNotepad **Failed**

Test Case Windows
Started at 6/27/2026, 8:51:36 PM

SampleNotepad **Failed**

Test Case Chrome
Started at 6/27/2026, 8:50:05 PM

SampleNotepad **Failed**

Test Case Chrome
Started at 6/27/2026, 8:46:21 PM

Problems Event Log Console Log Viewer Self-healing Insights

(1/1) test case 0 Passed 1 Failed 0 Error 0 Incomplete 0 Skipped [Troubleshoot With AI](#)

Test Cases/SampleNotepad 1m 2.273s

Test case Test Cases/SampleNotepad
Start time 2026-06-27 20:55:53.6 Duration 1m 2.273s
2 test steps 1 Passed 1 Failed

Fail Reason	Test Steps	Execution Information
hostName	jenncc - BCRCyberTesting	
hostAddress	192.168.1.20	
os	Windows 11 64bit	
userFullName	JR Clark	

APPENDIX 7: Desktop Testing Method Alternatives to Katalon Studio

Given Katalon Studio's documented Desktop testing failures, alternatives include two Python-based approaches that directly address the same testing objectives using different architectural layers. **pywinauto** operates by calling the Windows UI Automation API directly — no server required, minimal setup, and well-suited to lightweight scripted automation. **Appium + WinAppDriver with a Python client** mirrors the server/client architecture that Katalon Studio uses internally, but bypasses Katalon's GUI abstraction layer entirely, exposing the underlying Appium infrastructure. A third option, **FlaUI (.NET/NUnit)**, is notable because Katalon itself adopted it in version 10.3.0 as a replacement for WinAppDriver in its Desktop testing beta.

Criterion	Katalon Studio (Desktop)	pywinauto (Python)	Appium + WinAppDriver (Python)
Setup complexity	Low (GUI-based)	Low (pip install)	Medium (Appium server required)
Language	Groovy/Java hybrid	Python	Python
Architecture	GUI abstraction over WinAppDriver	Direct Windows UI Automation API	Server/client (Appium protocol)
AUT support	UWP, Win32	Win32, WPF, WinForms	UWP, Win32
Active development	Buggy in 11.2.1; FlaUI beta in 10.3+	Active (open source)	WinAppDriver stalled; Appium active
Execution result	❌ Failed (documented bugs)	✅ Viable	✅ Viable
Inspector tool	Spy utility (non-functional)	pywinauto inspect	Appium Inspector / Inspect.exe

Note: Scripts for both Python methods were developed and are available but were not executed as part of this lab submission due to the Katalon Studio failures occurring at the framework configuration level rather than the test design level. See: <https://claude.ai/share/5f8fbed6-1306-4217-99de-890ec6fa71f>

From: Claude.ai <https://claude.ai/share/228e35b5-6a53-4902-9202-932f44cf789b>

REFERENCES

Katalon. (2026a). *Desktop applications testing with FlaUI driver in Katalon Studio*.

<https://docs.katalon.com/katalon-studio/manage-projects/set-up-projects/windows-desktop-apps-testing/desktop-applications-testing-with-flaui-driver-in-katalon-studio>

Katalon. (2026b). *Sample Windows application test* [GitHub repository].

<https://github.com/katalon-studio/sample-windows-application-test>

Katalon. (2026). *Introduction to desktop app testing in Katalon Studio*.

<https://docs.katalon.com/katalon-studio/get-started/supported-applications-under-test-aut/introduction-to-desktop-app-testing-in-katalon-studio>

Katalon. (2026). *Sample API tests project in Katalon Studio*. <https://docs.katalon.com/katalon-studio/get-started/sample-projects/api/sample-api-project-in-katalon-studio>

Katalon. (2026). *Sample Android mobile project in Katalon Studio*.

<https://docs.katalon.com/katalon-studio/get-started/sample-projects/mobile/sample-android-mobile-project-in-katalon-studio>

Katalon. (2026). *Create and run Android test case in Katalon Studio*.

<https://docs.katalon.com/katalon-studio/get-started/sample-projects/mobile/mobile-create-and-run-android-test-case-in-katalon-studio>

Katalon Community. (2026). *Since Katalon 11.1.x, Windows.startApplication is broken*.

<https://forum.katalon.com/t/since-katalon-11-1-x-windows-startapplication-is-broken/189818>

Katalon. (2026). *Data-driven testing with RESTful web service requests*.

<https://docs.katalon.com/katalon-studio/data-driven-testing/data-driven-testing-with-restful-web-service-requests>

Sauce Labs. (2026). *SauceDemo*. <https://www.saucedemo.com>

Methodology and Use of AI Tools

Claude and CoPilot were consulted on how to find the right materials for testing; CoPilot specifically recommended the sample projects. Claude.ai was used to troubleshoot the Windows 11 configuration & provided the text describing the issues with creating a successful Desktop Demo (See Appendix 5 and Appendix 6). The analysis, observations, and conclusions reflect the author's own hands-on experience with the tool and tasks analyzed. All content has been reviewed for accuracy and represents the author's understanding of the results.

Reference to CoPilot LLM conversation on [Using Katalon's Sample API Project](#)

Reference to Claude LLM conversation on Katalon API testing: <https://claude.ai/share/f9861ee7-bbdf-4554-9e8d-be090db8f595>

Reference to CoPilot LLM conversation on Mobile tools implementation:
<https://copilot.microsoft.com/chats/gHYRtSCqj1SSZbqVPfty7>

Reference to Claude LLM conversation on draft and rubric compliance:
<https://claude.ai/share/8c081e96-342b-4f01-9f50-e65334531200>